AI Meets Aqua

Goldfish Stress Detection Web App
3-Week Detailed Workflow by Role

Product (web version): a user uploads a goldfish tank video. The system cleans the video, detects and tracks each goldfish, measures four behaviour signals (swimming speed, direction changes, isolation, vertical position), scores stress per fish and for the tank, highlights stressed fish with red boxes, and gives action-based suggestions. Stress history is saved per tank.

Built on: your SIP review deck and the AI Meets Aqua deck: YOLOv8 detection, DeepSORT tracking, the 4-metric stress decision engine (threshold 35), the Behaviour - Cause - Action alert table, stress history with a 7-day recycle bin, and the Fish Stress Index (FSI).

Scope: software only, goldfish only, one web app.

Role	People	Owns
Frontend	2 (Frontend 1 and Frontend 2)	Landing experience inspired by hackfest.dev, upload flow, results dashboard, history
Backend	1	API, database, jobs, suggestions engine, history and recycle bin, deployment, documentation, research paper
AI	2 (AI-1 and AI-2)	Goldfish detection and tracking (AI-1); behaviour features, stress scoring and evaluation (AI-2)

Part 1. Outline

1.1 End-to-end flow

- Create or choose a tank profile (volume, number of goldfish, water temperature).
- Upload a 30-60 second tank video with a few optional details (last water change, last feeding, camera view).
- Backend validates the file and creates a job; the UI shows live stage progress.
- AI pipeline: pre-process (glare, ripples, bubbles) - YOLOv8 goldfish detection - DeepSORT tracking (Fish_1, Fish_2, ...) - behaviour features - stress score.
- Report: tank stress score, per-fish scores, timeline, annotated video (green normal, red stressed), key moments, and Behaviour - Cause - Action suggestions.
- History: every analysis saved per tank, trend chart, FSI, alerts, and a 7-day recycle bin for deleted clips.

1.2 Architecture and stack

Layer	Choice	Owner
Frontend	Next.js (App Router) + TypeScript + Tailwind, Framer Motion and GSAP ScrollTrigger for motion, React Query, Recharts	Frontend 1 and 2
Backend API	FastAPI, Pydantic, SQLAlchemy, SQLite (PostgreSQL optional), background worker	Backend
AI pipeline	OpenCV, Ultralytics YOLOv8, deep-sort-realtime, NumPy/Pandas, scikit-learn, ffmpeg	AI-1 and AI-2
Storage	Local disk: uploads, outputs, trash (7-day recycle bin)	Backend
Deployment	Docker Compose; frontend on Vercel/Netlify, backend + AI on a container host or college GPU/CPU machine behind a tunnel	Backend + Frontend

1.3 Three-week milestones

Week	Goal	Demo at end of week
1 (Days 1-7)	Foundations: repo, frozen API contract, goldfish dataset and first detector, design system and landing hero, mock-data app shell	Upload a clip in the UI, see a complete mock report rendered from the API
2 (Days 8-14)	Core: real tracking and scoring, suggestions engine, history, full dashboard on real results	Real goldfish video analysed end to end with annotated output
3 (Days 15-21)	Evaluation, polish, deploy, tests, paper and documentation, final demo	Live deployed site, evaluation tables, paper draft, documentation pack

1.4 Team rules

- **Day 2: API contract frozen.** Frontend builds on a mock API, AI builds against one Python function. No one waits on anyone.
- Monorepo: `/frontend /backend /ai /docs`. Branches: main (protected), dev, feature branches. One review per pull request.
- 10-minute daily standup. Integration checkpoints on Day 7, Day 14 and Day 19. Feature freeze on Day 18.
- One shared board for tasks and one shared folder of goldfish test clips (normal and stressed) used by everyone.

Part 2. Goldfish Scope and Shared Contracts

2.1 What the system looks for in goldfish

Signal	Normal goldfish	Stress indicator	Needs view
Swimming speed	Calm gliding, steady pace	Very fast darting, or very slow and weak	Any
Direction changes	Smooth turns	Frequent sharp turns, glass hitting (erratic)	Any
Isolation	Stays loosely with tank-mates	Alone in a corner for a long period	Any (2+ fish)
Vertical position	Mid-water	Gasping at surface, or inactive at the bottom	Side/front view

Filming guide shown in the app: side or front view of the tank, steady camera, 30-60 seconds, 720p or higher, even lighting, 1-5 goldfish. If a top view is uploaded, the vertical-position signal is switched off and the remaining weights are re-balanced automatically, and the report says so.

2.2 Stress scoring (v1, from your decision engine)

Per fish, points are added: speed outside the normal range +10, isolation +15, surface gasping +20, inactive at the bottom +15, optional 'eating normally' -5. A fish with 35 points or more is flagged stressed (red box); below that it stays green. Tank score (0-100) = 0.6 x average fish score + 0.4 x highest fish score. Levels: **Normal 0-34, Moderate 35-59, High 60-100**. Per-tank baselines and data-driven thresholds replace the starting values once AI-2 has analysed the dataset.

2.3 Behaviour - Cause - Action rules (editable file, each with a citation)

Detected behaviour	Possible cause	Suggested action
Goldfish gasping near the surface	Low oxygen, high ammonia or warm water	Turn on the air pump or increase aeration; check temperature; test ammonia
Fast swimming or hitting the glass	Poor water quality or sudden disturbance	Change about 25% of water with temperature-matched, dechlorinated water; reduce light and noise
Fish alone at the bottom	Stress or early infection	Observe Fish_N for 24 hours; check for spots or clamped fins; move to a quarantine container if signs appear
All fish inactive	Water temperature or feed issue	Check water temperature (goldfish are comfortable around 18-24 C); check feed quality and amount

2.4 API endpoints

Method	Endpoint	Purpose
POST / GET	/api/v1/tanks	Create and list tank profiles
POST	/api/v1/videos	Upload video with tank_id and details; returns job_id
GET	/api/v1/jobs/{id}	Status, stage name and progress %
GET	/api/v1/results/{id}	Full result JSON (below)
GET	/api/v1/results/{id}/video	Annotated video; key frames under /frames
GET	/api/v1/history?tank_id=	Past analyses and trend data
DELETE / POST	/api/v1/results/{id} and /restore	Move to recycle bin (7 days) and restore
GET	/api/v1/insights?tank_id=	FSI and simple pattern messages
GET	/api/v1/results/{id}/report.pdf	Downloadable report
GET	/health	Deployment health check

2.5 Result JSON and AI function signature

```
{ "job_id": "a1b2", "tank_score": 62, "level": "High", "view": "side",
  "fish": [ { "id": "Fish_1", "score": 48, "stressed": true, "flags":
["surface_gasping","fast_swimming"],
    "metrics": { "speed": 41.0, "direction_change_rate": 0.34, "isolation_ratio": 0.10,
"surface_ratio": 0.42 } } ],
  "timeline": [ { "t": 0, "score": 30 }, { "t": 5, "score": 52 } ], "threshold": 35,
  "key_moments": [ { "t": 12.5, "fish": "Fish_1", "behaviour": "surface_gasping", "frame_url": "..."
} ],
  "suggestions": [ { "priority": "high", "behaviour": "...", "cause": "...", "action": "..." } ],
  "warnings": [], "disclaimer": "Behavioural indicator to support caretaker decisions" }

analyze_video(video_path, meta, progress_cb) -> dict # returns the JSON above
```

Part 3. Frontend Workflow (2 people)

3.0 Design direction inspired by hackfest.dev

hackfest.dev tells one story through scrolling: a branded loading screen with an asset percentage, a glowing logo hero with a 'Dive Deeper / Scroll Down' cue, a rich textured dark-teal background, a timeline of 'Voyage Logs', a downloadable brochure card ('Captain's Log'), an FAQ accordion, a compass link, and a footer with sponsors. AI Meets Aqua translates that structure into an original underwater experience (original artwork, code and copy; only the structure and feel are borrowed).

Hackfest pattern	AI Meets Aqua version
Loading screen with percentage	'Descending...' loader with water filling to 100% while hero assets preload
Glowing logo hero, 'Dive Deeper', 'Scroll Down'	Glowing AI Meets Aqua mark over animated water, light rays and bubbles; goldfish swims across; 'Dive Deeper' cue
Journey theme, compass	Depth meter fixed at the side (0 m to 10 m) that changes as you scroll; the background gets deeper and darker
Voyage Logs timeline	'Dive Log' timeline: Upload - Detect - Track - Analyse - Act
Tracks section	Four behaviour signal cards (speed, direction, isolation, vertical position)
Captain's Log brochure card	'Project Brief' card linking to the project deck/report
FAQ accordion	'Answers from the tank' accordion
Textured teal background, glow accents	Deep teal gradient with subtle grain, aqua and goldfish-orange accent glow
Navbar with highlighted Register button	Navbar: Home, How It Works, Analyse, History, About + highlighted 'Analyse Video' button

Split: Frontend 1 owns the design system and the public landing experience (all the motion and storytelling). **Frontend 2** owns the working application: tank profiles, upload, processing, results dashboard, history and alerts. Both use the same components and tokens.

3.1 Frontend 1: Design system and landing experience

Days	Tasks	Output
1	Set up Next.js + TypeScript + Tailwind + Framer Motion + GSAP ScrollTrigger in the monorepo with Frontend 2. Moodboard from hackfest.dev; pick palette (deep teal, aqua glow, goldfish orange), one display font and one body font. Define design tokens.	Project running, tokens file
2	Build the base kit: Button, Card, Badge, Section header, Glow wrapper, Accordion, Navbar, Footer. Document in a small component page.	Shared component library
3	Loading screen: 'Descending...' with percentage tied to real asset preloading; exit transition into the hero.	Loader live
4-5	Hero: animated water (canvas or CSS), light rays, floating bubbles, glowing logo, goldfish swimming loop (SVG or Lottie), headline, 'Dive Deeper' and scroll cue, primary CTA to Analyse.	Hero complete
6	Depth meter: fixed side gauge tied to scroll position; background colour shifts deeper per section.	Scroll depth system
7	Checkpoint 1. Problem section (stress spreads through shared water, manual observation misses early signs, key figures from your deck) with count-up numbers.	Checkpoint 1 passed
8	'Dive Log' timeline section: five steps, each reveals on scroll with a small illustration.	Timeline section
9	Four behaviour signal cards with micro-animations (fast dart, erratic path, lone fish, surface gasp). Stress level explainer (Normal / Moderate / High).	Signals section
10	Behaviour - Cause - Action section as flip cards (from the rules table). SDG 6, 12, 14 strip with short justifications.	Action and SDG sections
11	'Answers from the tank' FAQ accordion, 'Project Brief' card, team section with institution details, footer.	Landing page complete

Days	Tasks	Output
12	Sample report preview embedded in the landing page (static sample result) with a button to try your own video.	Preview section
13	Responsive pass (mobile, tablet, desktop), reduced-motion support, image optimisation, lazy loading; Lighthouse performance and accessibility check.	Responsive and fast
14	Checkpoint 2. Fix visual issues found with real data on Frontend 2's pages; align spacing and typography across the whole site.	Checkpoint 2 passed
15-16	Page transitions between landing and app, About and Contact pages, 404 page, SEO titles and social preview image, favicon.	Polished navigation
17	Cross-browser testing (Chrome, Edge, Firefox, Safari), final animation tuning, optional dark/light refinements.	Browser-ready
18	Feature freeze. Bug bash with the team; fix visual and responsive bugs.	Stable UI
19-20	Screenshots and UI-flow diagram for the paper and documentation; record a 2-minute screen demo.	Assets for paper and demo
21	Final regression on the deployed site; demo rehearsal.	Ready to present

3.2 Frontend 2: Application (tank, upload, results, history)

Days	Tasks	Output
1	Set up the app routes with Frontend 1 (/analyse, /results/[id], /history, /tanks). React Query, Axios client, types generated from the OpenAPI schema, env config.	App shell
2	Tank profile form and list (name, volume, number of goldfish, water temperature). Upload page skeleton against the mock API. Add the goldfish filming guide panel.	Tank + upload skeleton
3-4	Upload page: drag and drop, video preview, client-side checks (type, size, duration), optional details (last water change, last feeding, camera view), upload progress bar with cancel.	Upload flow complete
5	Processing screen: polls job status every 2-3 seconds, shows stage names (Cleaning video, Detecting goldfish, Tracking fish, Analysing behaviour, Preparing report) with progress, themed animation, failure and retry states.	Processing screen
6	Stress gauge (0-100 with Normal / Moderate / High bands), level badge, per-fish cards (Fish_1... with red or green status) using mock JSON.	Gauge + fish cards
7	Checkpoint 1: connect to the real backend mock endpoints; fix contract mismatches with Backend.	Checkpoint 1 passed
8-9	Results dashboard: timeline chart (Recharts) with the 35-point threshold line and a fish selector, behaviour metric cards with plain-language explanations, key-moment gallery with timestamps.	Dashboard v1
10	Annotated video player (green and red boxes from the backend render) with click-to-seek from timeline points and key moments.	Video-synced results
11	Suggestions panel in Behaviour - Cause - Action format with priority tags and a 'mark as done' control; disclaimer text.	Suggestions UI
12	History page: analyses by tank, tank score trend chart, FSI card, insight messages.	History page
13	Recycle bin: delete moves a result to the bin with a 7-day countdown, restore button, empty state. Alert settings UI (dashboard pop-up on by default, optional email).	Recycle bin + alerts UI
14	Checkpoint 2: full end-to-end on real analyses; collect and fix bugs. Dashboard pop-up when a stressed goldfish is found.	Checkpoint 2 passed
15	Download report button (PDF endpoint), loading skeletons, empty states, error boundaries.	Complete states
16	Insights panel (pattern messages such as stress rising after low-temperature entries) using the insights endpoint. Demo mode button that opens a pre-analysed sample.	Insights + demo mode
17	Playwright tests: upload - processing - report happy path, invalid file, failed job, recycle bin restore. Accessibility checks.	Automated tests
18	Feature freeze. Bug bash with the team; fix application bugs.	Stable app

Days	Tasks	Output
19-20	Production build, deploy with Frontend 1, point to live backend, fix CORS or URL issues; screenshots for documentation.	Live frontend
21	Final regression on the deployed site; demo rehearsal.	Ready to present

Part 4. Backend Workflow (1 person)

Mission: own the API, data, job pipeline, suggestions engine, deployment, documentation and the research paper. Keep a short decisions log from Day 1; it becomes the Methodology section of the paper.

Days	Tasks	Output
1	Create the monorepo, branch rules, README, task board. FastAPI project structure (routers, services, models, schemas), virtualenv, requirements. Kick-off: assign owners and confirm the plan.	Repo live
2	Freeze the API contract (Part 2) as Pydantic schemas with OpenAPI. Publish a mock API that returns a realistic goldfish result so both frontend developers can start immediately. Share a Postman collection.	Mock API at /docs
3	Database with SQLAlchemy: tanks, videos, jobs, results, alerts, trash. Initial migration.	DB models
4	Tank endpoints. Upload endpoint: validate file type (mp4, mov, avi), size cap, duration cap via ffprobe/OpenCV, store with a UUID name, create job.	Upload working
5	Job runner: background worker that calls a stub analyze_video, updates status, stage and progress, handles timeouts and failures. GET /jobs/{id}.	Async job lifecycle
6	Config through .env, CORS, logging, a consistent error format, request size limits.	Stable base
7	Checkpoint 1 with frontend and AI. Start the paper skeleton and reference list (15-20 papers on fish behaviour, stress indicators and video tracking).	Checkpoint 1 passed
8-9	Integrate the real AI package (ai/pipeline.py): worker calls analyze_video, progress callback updates the job, outputs saved (result JSON, annotated video, key frames). Configure CPU/GPU and memory limits.	Real results flow
10	Suggestions engine: load the Behaviour - Cause - Action rules file (YAML), map each flagged behaviour of each fish to suggestions with priority, include Fish ID in the text. Add citations to every rule for the paper.	Suggestions in API
11	History and trend endpoints, annotated video serving with range requests, key frame serving, pagination.	History + media
12	Recycle bin: soft delete to a trash folder, restore endpoint, scheduled purge job after 7 days. Storage cleanup for raw uploads.	Smart storage
13	Alerts: create an alert record when a fish is flagged; endpoint for the dashboard pop-up; optional email alert with a simple SMTP service. Insights endpoint with FSI (average stress index per tank over time) and simple pattern messages.	Alerts + FSI
14	Checkpoint 2: end-to-end runs on at least 5 goldfish clips. Pytest suite for upload validation, job lifecycle, suggestions mapping, recycle bin. Draft Related Work and Methodology.	Checkpoint 2 passed
15	Dockerfile for backend plus AI dependencies, docker-compose for local run. Decide the host for inference (container host, college machine with a tunnel, or GPU notebook service) and test processing time on it; set a clip limit that keeps analysis quick.	Containers build
16	Deploy the backend, connect the live frontend, HTTPS, environment variables, smoke tests. Prepare pre-analysed sample results for the demo mode.	Live API
17	PDF report export (reportlab): score, per-fish table, timeline chart, suggestions. Security pass: file validation, size limits, safe paths, rate limiting.	Report + hardening
18	Feature freeze. Bug bash with the team; record processing-time and resource numbers for the paper.	Stable build
19	Documentation pack: README, setup guide, API documentation (OpenAPI export), architecture diagram, user manual with screenshots, deployment guide. Paper: Implementation, Results and Discussion.	Docs v1
20	Paper: Abstract, Introduction, Conclusion, limitations, formatted references, figures and tables from the AI team. Team review.	Paper draft complete
21	Final proofread, similarity check, demo script and rehearsal, backup of repo and demo video.	Submission ready

Research paper outline (goldfish focus)

- Abstract and keywords
- 1. Introduction: stress spreads quickly in shared water, manual observation misses early signs, need for affordable video-based monitoring of ornamental fish
- 2. Related Work: computer vision in aquaculture, fish behaviour and stress indicators, tracking methods, commercial systems and their gaps
- 3. System Architecture (web app diagram)
- 4. Methodology: goldfish dataset, pre-processing, YOLOv8 detection, DeepSORT tracking, four behaviour signals, stress decision engine, suggestions rules
- 5. Implementation: stack, API, job pipeline, history and recycle bin
- 6. Experiments and Results: detection (precision, recall, mAP), tracking (ID switches, IDF1), stress classification (accuracy, F1, ROC-AUC), ablation per signal, runtime, label-noise robustness
- 7. Discussion and Limitations: reflections, plants, camera view, tank variety
- 8. Conclusion and Future Work: real-time camera feed, sensor fusion, multi-species, mobile alerts
- References

Part 5. AI Workflow (2 people)

AI-1 owns goldfish data, pre-processing, detection and tracking. **AI-2** owns behaviour features, stress scoring, evaluation and explainability. AI-2 builds on synthetic trajectories until AI-1's tracks are ready, then switches to real tracks. They meet every day and share one repository folder (/ai).

5.1 AI-1: Data, pre-processing, detection and tracking

Days	Tasks	Output
1	Inventory goldfish footage: your labelled clips (normal and stressed), field-visit recordings, public goldfish and fish datasets with usable licences. Create a data log. Set up Colab/Kaggle GPU and Ultralytics.	Data log + environment
2	Run pretrained YOLOv8n on goldfish frames and record where it struggles (reflections, orange decorations, plants, fancy goldfish shapes).	Baseline + failure list
3-4	Extract frames (about one per 0.5 s), remove near-duplicates, annotate goldfish boxes in Roboflow/CVAT (target 800-1500 frames across different tanks, angles and lighting). Split train, validation and test by video, not by frame. Define augmentations (brightness, blur, flip, mosaic).	Annotated dataset
5-6	Fine-tune YOLOv8n and YOLOv8s (single class: goldfish). Track experiments in a sheet or Weights and Biases; compare image sizes and augmentations. Report precision, recall, mAP50, mAP50-95. Target mAP50 of 0.85 or higher, improving on the current 75%.	Best weights
7	Integrate DeepSORT (deep-sort-realtime) with the detector: unique IDs Fish_1, Fish_2... Provide detect_and_track(video) returning tracks.json. Checkpoint 1 with AI-2 and Backend.	Checkpoint 1 passed
8	Pre-processing module: brightness normalisation and CLAHE, glare and reflection suppression, motion/size filters for bubbles and ripples. Produce before-and-after comparisons.	Clean-video module
9	Tune the tracker (max age, minimum hits, appearance embedder) for goldfish that look alike. Count ID switches and IDF1 on 5-10 manually checked clips. ByteTrack kept as an alternative if it scores better.	Tracking tuned
10	Annotated video renderer: green boxes for normal fish, red boxes for stressed fish (flags supplied by AI-2), ID labels, short trails, H.264 output via ffmpeg for browser playback; key frame saver.	Annotated output
11	Speed work: frame stride, image size, half precision on GPU, batching. Benchmark CPU and GPU for a 60-second clip and set the recommended clip length.	Runtime table
12	Package as ai/tracking.py with a progress callback and clear exceptions; unit tests; handoff notes for Backend.	Module delivered
13	Robustness tests: low light, glare, cloudy water, plants, 1 vs 5 goldfish, different goldfish varieties (common, fancy), camera angles. Keep a failure log with screenshots.	Robustness report
14	Checkpoint 2: run the full pipeline through the backend on several clips, fix integration errors.	Checkpoint 2 passed
15-16	Final evaluation: precision-recall curve, confusion cases, mAP, tracking metrics, runtime. Failure gallery with explanations.	Evaluation tables
17	Fixes from integration; pin library versions; write the model card (data, metrics, limits).	Model card
18	Feature freeze; bug bash.	Stable models
19	Hand figures to the paper owner: training curves, PR curve, sample annotated frames, before/after pre-processing, tracking examples.	Paper figures
20	ai/README: how to train, how to run inference, expected results. Review the detection and tracking parts of the paper.	Reproducible repo
21	Viva preparation: detection and tracking questions, demo rehearsal.	Ready to present

5.2 AI-2: Behaviour features, stress scoring and evaluation

Days	Tasks	Output
1	Review literature on goldfish stress behaviour (surface gasping, glass hitting, erratic darting, hiding, lethargy at the bottom, isolation). Confirm the four core signals from your deck.	Indicator list
2	Write the feature specification: speed in body-lengths per second, direction-change rate (turning angle above a threshold per second), isolation ratio (time far from the nearest tank-mate), vertical position (surface and bottom zones as fractions of tank height), activity ratio. Agree the JSON with Backend.	Feature spec
3-4	Implement feature extraction on tracks (per fish and per 5-second window). Test on synthetic trajectories: calm glide, circling, jitter, static, lone fish, surface hovering.	Feature module + tests
5	Scoring v0 from the decision engine: speed +10, isolation +15, surface gasping +20, bottom inactivity +15, optional eating normally -5; stressed at 35 points or more; tank score formula and levels from Part 2.	Scoring v0
6	Per-tank baseline (normalise against a calm reference when available) and goldfish default ranges. View handling: top view disables vertical position and re-balances weights; add a warning in the output.	Baselines + view logic
7	Checkpoint 1: compute_scores(tracks, meta) runs on mock tracks and returns the exact contract JSON. Prepare an evaluation list of 30 or more normal and stressed clips.	Checkpoint 1 passed
8-9	Connect to AI-1's real tracks. Build the windowed timeline with smoothing, per-fish flags and key-moment detection with behaviour labels (these labels feed the backend suggestions engine).	Real scoring
10	Build a clip-level feature table from your labelled clips; study distributions of normal vs stressed; set thresholds from the data (percentiles) and compare with the starting values.	Data-driven thresholds
11	Train a small classifier (logistic regression, random forest, XGBoost) on clip-level features; cross-validate; report accuracy, precision, recall, F1, ROC-AUC. Keep the rule-based score as the explainable output and use the classifier as a confidence check.	Hybrid model
12	Edge cases: no fish found, one fish (isolation not applicable), fish leaving the frame, very short clips, poor video quality. Return clear warnings rather than errors.	Graceful handling
13	Sensitivity analysis on weights and thresholds. Label-noise experiment: flip 5-10% of training labels and show the model still learns the majority pattern (matches your deck's robustness claim).	Robustness results
14	Checkpoint 2: freeze score formula v1 and release analyze_video in ai/pipeline.py.	Checkpoint 2 passed
15-16	Final evaluation: confusion matrix, per-signal ablation, rule-based vs hybrid comparison, error analysis on missed and false alarms.	Evaluation tables
17	Explainability: a one-line 'why flagged' text per fish (for example, 'Fish_2 stayed near the surface for 40% of the clip').	Explanations in output
18	Feature freeze; bug bash.	Stable scoring
19	Hand tables and plots to the paper owner: ablation, confusion matrix, ROC curve, sample timelines.	Paper figures
20	Review the Methodology and Results sections for technical accuracy; write the scoring section of ai/README.	Reviewed paper sections
21	Viva preparation: scoring, thresholds and evaluation questions; demo rehearsal.	Ready to present

Part 6. Combined Timeline

Day	Frontend 1	Frontend 2	Backend	AI-1	AI-2
1-2	Setup, tokens, UI kit	App shell, tanks, upload skeleton	Repo, frozen contract, mock API	Dataset, baseline YOLO	Literature, feature spec
3-6	Loader, hero, depth meter	Upload, processing, gauge	DB, upload, job runner	Annotate, train, tune	Features, scoring v0
7	Checkpoint 1	Checkpoint 1	Checkpoint 1	Checkpoint 1	Checkpoint 1
8-13	Landing sections, responsive	Dashboard, video sync, history, bin	Real AI, suggestions, history, bin, alerts	Pre-processing, tracking, render, speed	Real scoring, thresholds, hybrid model
14	Checkpoint 2	Checkpoint 2	Checkpoint 2	Checkpoint 2	Checkpoint 2
15-17	Transitions, SEO, browsers	Report, insights, tests	Docker, deploy, PDF, security	Evaluation, model card	Evaluation, explainability
18	Freeze + bug bash	Freeze + bug bash	Freeze + bug bash	Freeze + bug bash	Freeze + bug bash
19-21	Demo video, screenshots	Deploy, final tests	Docs, paper, demo	Figures, README	Figures, review, viva

Part 7. Risks and Fallbacks

Risk	Fallback
Goldfish look alike, IDs switch	Tune DeepSORT embedder, try ByteTrack, report IDF1 honestly; keep clips short
Glass reflections and bubbles cause false detections	Pre-processing filters plus hard-negative frames added to training
Limited stressed footage	Augmentation, label-noise experiment, per-tank baselines, contrast clips (calm vs disturbed)
Inference speed on the hosting machine	YOLOv8n, frame stride, 30-60 second clip limit, demo mode with pre-analysed samples
Integration surprises	Frozen contract, mock API from Day 2, checkpoints on Days 7, 14 and 19
Schedule pressure	Scope can be trimmed in this order: insights panel, email alerts, PDF export, landing extras. The core always stays: upload, tracking, per-fish stress score, timeline, suggestions, history

Part 8. Definition of Done

- A user can upload a goldfish video on the deployed site and receive a tank stress score, per-fish scores with red and green boxes, timeline, key moments and Behaviour - Cause - Action suggestions.
- History per tank, 7-day recycle bin and alerts work.
- Detection, tracking and stress classification results are reported with metrics and limitations.
- Repo has README, setup guide and API docs; tests pass; demo mode is available.
- Paper draft, documentation pack, demo video and slides are ready.